

Full Length Article

NEU: Continuous-time memory evolution for temporal interaction graph networks

Yang Meng^a, Yuting Liu^{a,b,*}^a School of Mathematics and Statistics, Beijing Jiaotong University, Beijing, 100044, China^b Beijing Key Laboratory of Biological Big Data and Topological Statistics, Beijing, 100044, China

ARTICLE INFO

Keywords:

Temporal interaction graphs
Memory evolution
Continuous-time modeling
Ordinary differential equations

ABSTRACT

Graph neural networks (GNNs) have become a cornerstone for modeling complex relational data, yet most canonical architectures assume the graph topology is static. In real-world temporal interaction graphs (TIGs), however, nodes and edges evolve continuously. Recent memory-augmented temporal GNNs address this by maintaining a learnable memory for every node, which is discretely updated only when an interaction occurs. Although memory based models such as TGN and TGAT include time encoding as a marker representing the update time of node memory, the elapsed time merely serves as a static input feature that modulates the next embedding update. The memory itself is still assumed to remain unchanged between two events, implying that the node state evolves through a sequence of instantaneous jumps. This event-driven assumption neglects two key temporal phenomena: (1) natural decay—a node's latent state gradually loses relevance during long inactivity; and (2) short-term drift—the latest neighbour exerts a residual influence that fades smoothly rather than abruptly. Consequently, memories of sparsely active nodes become stale. We can only obtain the memory of the node at the time of the event, but cannot obtain the memory at any given time. The discrete-update paradigm cannot infer a node's state at arbitrary intermediate timestamps, limiting both accuracy and temporal consistency. To tackle these limitations, we introduce the **Natural Evolution Unit (NEU)**, a module that inserts a continuous-time memory evolution stage before the embedding read-out. NEU treats the time difference not as an auxiliary input but as a dynamic driver of a learnable ordinary differential equation, enabling memory to drift and decay smoothly between events. This brings two benefits: (1) node states become continuously queryable at arbitrary timestamps, providing continuous time inferability and interpretability; and (2) our modeling of time differences has been upgraded from “static input” to “dynamic evolution driven”, allowing the ODE-driven dynamics to dominate the temporal modeling and reducing dependence on learnable time encoding. Therefore, we adopt a fixed time encoding to control the stability of training, thereby further improving the experimental results. We conducted comprehensive experiments on five public datasets, and the results showed that NEU consistently improved AUC and AP on the strongest memory based baseline. This indicates that our method is crucial for mitigating memory aging and enhancing long-term prediction, offering a simple yet effective new perspective for representation learning on dynamic graphs.

1. Introduction

Over the past few years, graph representation learning has made significant progress in fields like social networks, recommender systems, biological networks, and traffic systems (Battaglia et al., 2018; Bronstein et al., 2017; Dai et al., 2016; Grover & Leskovec, 2016; Hamilton et al., 2017; Jiang & Luo, 2022; Ju et al., 2024; Kang & McAuley, 2018; Souza et al., 2022; Tang & Wang, 2018; Wu et al., 2017). This benefits from the development of graph neural networks (GNNs), which use message passing mechanisms to capture local neighborhood information and en-

able downstream tasks such as node classification (Kipf & Welling, 2017; Veličković et al., 2018), graph classification (Gilmer et al., 2017), and link prediction (Zhang & Chen, 2018).

Many real-world systems can be formulated as temporal interaction graphs (TIGs), with nodes and edges constantly evolving over time. Early methods of applying static GNNs to such dynamic environments by taking graph snapshots (Dunlavy et al., 2011; Liben-Nowell & Kleinberg, 2003; Sankar et al., 2020; Yu et al., 2019) often leads to suboptimal performance, as these methods fail to capture crucial temporal dependencies and fine-grained interaction patterns (Xu et al., 2020).

* Corresponding author.

E-mail addresses: 23121730@bjtu.edu.cn (Y. Meng), ytliu@bjtu.edu.cn (Y. Liu).<https://doi.org/10.1016/j.neunet.2026.108998>

Received 14 January 2026; Received in revised form 27 March 2026; Accepted 13 April 2026

Available online 15 April 2026

0893-6080/© 2026 Elsevier Ltd. All rights are reserved, including those for text and data mining, AI training, and similar technologies.

This limitation has motivated the rise of temporal interaction graph learning (Kazemi et al., 2020; Rossi et al., 2020; Zhang et al., 2021), where edges (and sometimes nodes) arrive as time-stamped events. Depending on how time is modeled, existing methods can be broadly categorized into discrete-time approaches, which model graph dynamics via sequential snapshots (Pareja et al., 2020; Trivedi et al., 2019), and continuous-time approaches, which treat events as timestamped interactions (Kumar et al., 2019; Ma et al., 2020; Xu et al., 2020). The latter has gained increasing attention due to its flexibility and ability to handle asynchronous events with fine granularity (Nguyen et al., 2018a; Trivedi et al., 2019).

A representative continuous-time framework is TGN (Rossi et al., 2020), which combines message passing with memory modules to track evolving node states and achieves state-of-the-art results on several benchmarks.

Subsequent works seek to improve various limitations of TGN. For example, TIGER (Zhang et al., 2023) introduces restart mechanisms to improve temporal embeddings in long sequences. In addition, other advanced methods that are not based on memory architecture have also emerged. GraphMixer (Cong et al., 2023) simplifies the architecture by discarding attention and recurrence while retaining strong performance through MLP-based aggregation. DyGFormer (Yu et al., 2023) adopts transformer-based modules to better capture long-range dependencies. FreeDyG (Tian et al., 2024) analyzes dynamic graphs from a frequency-domain perspective and proposes a node interaction frequency encoding module that both explicitly captures the proportion of common neighbors and the frequency of the interaction of the node pair. These methods reflect increasing interest in modeling long-term temporal patterns and simplifying dynamic graph learning pipelines.

Despite these advancements, two key challenges remain open:

(1) Representation Instability. In dynamic graph modeling tasks, node representations are typically updated only at the moments when interactions occur. However, this assumption overlooks a critical characteristic of real-world systems: nodes do not remain static between events. Even in the absence of direct interactions, a node's internal state—such as memory, preference, or influence—may continue to evolve naturally over time due to external signals, latent processes, or intrinsic decay.

(2) Temporal Discontinuity. Existing memory-based temporal GNNs can only return node states at event timestamps. They lack the ability to infer a node's state at an arbitrary query time t_q , which is often required by downstream tasks. As a result, the model must rely on stale memories produced at the last interaction, leading to discontinuous latent trajectories and poor temporal consistency.

In real-world systems, a node's internal state often evolves continuously even without discrete interactions. For example, a user's preference may drift gradually in a social network; a company's risk score changes over time in financial graphs; and a communication channel's condition decays between transmissions. If a model updates node states only during events, its representation quality will overly depend on interaction frequency, failing to capture the intrinsic temporal dynamics. Thus, accurately recovering node states at any intermediate timestamp is essential for robust temporal reasoning.

These limitations motivate our design of the **Natural Evolution Unit (NEU)**, a module that models the continuous evolution of node memories between events. After each interaction, a node's memory is updated and then evolves smoothly based on NEU containing learnable parameters. This mechanism enables the model to infer node states at any timestamp, ensuring temporally consistent and up-to-date representations even in sparse-event regimes or when long time gaps exist between interactions.

In order to achieve the continuous evolution of memory mentioned by NEU, we adopted an ODE based method. This ODE takes into account the influence of the node's previous memory and the memory of its nearest neighbors. Several works formulate message passing as differential equations on static graphs. Graph Neural ODEs (GNODE) (Poli et al.,

2019) models ODE between layers of a neural network, achieving continuity between layers. The resulting neural ODE can be integrated to any desired horizon, producing arbitrarily deep embeddings in a single forward pass. GRAND (Chamberlain et al., 2021) and GRAND++ (Thorpe et al., 2022) view heat diffusion as a graph ODE, while PDE-GCN (Eliasof et al., 2021) frames convolution as a learnable partial differential equation. Although elegant, these models aim to achieve layer-wise continuity and assume fixed topology. More importantly, they operate on raw features rather than the memory states used by event-driven TIG models, making them unsuitable for continuous-time memory evolution in dynamic graphs. We are motivated by the differential equation based models mentioned above and also adopt a custom ODE to represent the natural evolution of nodes' memories, although we cannot directly use these existing ODEs.

Our main contributions can be summarized as follows:

- We propose a **Natural Evolution Unit (NEU)** for the previous memory based model, which simulates the continuous state evolution of the internal state (memory) of nodes within event intervals. This mechanism enables the model to infer node states at any timestamp, ensuring temporal continuity and consistency, significantly enhancing the expressive power of the time graph network.
- For this NEU, we propose a continuous time memory evolution mechanism based on ordinary differential equations (ODEs). In addition, we have designed a natural evolutionary unit based on MLP as a lightweight alternative. This mechanism models the evolution of a node's own memory and the sustained influence of its neighbors.
- The ODE form we mentioned in NEU can be written as a closed form solution, achieving very low computational cost.
- Multiple experiments on six benchmark datasets have shown that our method outperforms the baseline, demonstrating the effectiveness of introducing NEU.

2. Related works

Continuous-time dynamic graph (CTDG) models aim to capture the evolving patterns of node and edge interactions over continuous time, without relying on fixed time intervals. Early works such as CTDNE (Nguyen et al., 2018b) encode temporal information by constraining random walks with timestamps, enabling preservation of temporal ordering in node sequences. DyRep (Trivedi et al., 2019) and MRDGNN (Duan et al., 2023) extend this by jointly learning node embeddings and event intensity functions via a temporal point process framework. TGAT (Xu et al., 2020) introduces temporal attention and time encoding to model the influence of temporal neighbors, while TGN (Rossi et al., 2020) proposes a memory-based architecture where each node maintains an external memory that is updated only when events occur. These models have laid the foundation for learning expressive representations in event-driven environments. Among CTDG methods, memory-based models introduce external memory modules to retain long-term historical states for each node, enabling more informed event-driven updates. Building on the memory-based framework, subsequent research has explored more advanced mechanisms to enhance memory usage. For example, APAN (Wang et al., 2021a) decouples memory updates from neighbor aggregation through an asynchronous mailbox mechanism, improving scalability in streaming scenarios. NAT (Luo & Li, 2022) proposes a GPU-friendly memory system based on dictionary indexing to enable fast and parallel updates. TIGER (Zhang et al., 2023) introduces a dual-memory module that distinguishes between short-term and long-term memory to better reflect temporal heterogeneity. GDCF (Han et al., 2023) focuses on crowd flow modeling and adopts a memory module to improve update speed. PRES (Su et al., 2024) applies Gaussian mixture modeling to forecast memory evolution based on historical gradients, thereby introducing temporal smoothness into the memory update process.

Beyond the above methods, there exists a type of modeling approach in the field of graph neural networks that utilizes ODE, including the methods mentioned in the previous section (Chamberlain et al., 2021; Eliasof et al., 2021; Poli et al., 2019). In addition, there are some methods that combine ODE with more complex diffusion dynamics (Choi et al., 2023; Eliasof et al., 2024; Li et al., 2024; Wang et al., 2021b), as well as methods that apply ODE to spatio-temporal graphs (Fang et al., 2021). These models formulate ODEs between layers of a neural network, thereby achieving a continuous-depth formulation of message passing. In other words, the ODE governs how node representations diffuse across the graph structure when stacking multiple layers, effectively modeling the propagation process itself.

In contrast, our work applies ODEs to model the latent memory state of each node between interactions. Rather than treating propagation or embeddings as continuous dynamics, we insert an ODE-based evolution step directly into the memory module. This design enables natural decay and short-term drift of memory in inactive periods, yielding fresher embeddings while maintaining compatibility with event-driven updates. Hence, NEU provides a fundamentally different use of ODEs: not to replace message passing or embedding propagation, but to refine temporal memory dynamics at the representation level.

3. Preliminaries

3.1. Deep learning on static graphs

We first revisit the standard message passing paradigm in static graphs. A static graph is denoted as $G = (\mathcal{V}, \mathcal{E})$, where $\mathcal{V} = \{1, 2, \dots, n\}$ represents the set of nodes, and $\mathcal{E} \subseteq \mathcal{V} \times \mathcal{V}$ denotes the set of edges. Each node $i \in \mathcal{V}$ is associated with a feature vector $\mathbf{x}_i \in \mathbb{R}^{d_v}$, and each edge $(i, j) \in \mathcal{E}$ may carry an edge feature $e_{ij} \in \mathbb{R}^{d_e}$.

Graph neural networks (GNNs) learn node embeddings by aggregating information from local neighborhoods. Let $\mathcal{N}_i$ denote the set of neighbors of node i . Then the embedding of node i can be computed as:

$$\mathbf{z}_i = F\left(\mathbf{x}_i, \sum_{j \in \mathcal{N}_i} \text{MSG}(\mathbf{x}_i, \mathbf{x}_j, e_{ij})\right). \quad (1)$$

Here, $\text{MSG}(\cdot)$ defines the message function that combines features of the source node, its neighbor, and the connecting edge. The function $F(\cdot)$ aggregates the messages together with the node's own feature and produces the final representation.

3.2. Temporal interaction graphs

We model dynamic graphs as temporal interaction graphs (TIGs), defined as a sequence of time-stamped interaction events:

$$\mathcal{G} = \{EV_k = (u_k, v_k, t_k, e_k)\}_{k=1}^M, \quad (2)$$

where each event EV_k denotes an interaction between a source node $u_k \in \mathcal{V}$ and a destination node $v_k \in \mathcal{V}$ at time $t_k \in \mathbb{R}_+$, and $e_k \in \mathbb{R}^{d_e}$ is the edge feature. Each node $i \in \mathcal{V}$ is associated with a possibly time-varying feature $\mathbf{x}_i(t) \in \mathbb{R}^{d_v}$. If features are not provided, zero vectors are assumed.

The history of node i up to time t is defined as:

$$\mathcal{H}_i(t) = \{(i, j, \tau, e_{ij} \in \mathcal{G} \mid \tau < t) \cup \{(j, i, \tau, e_{ji}) \in \mathcal{G} \mid \tau < t\}, \quad (3)$$

where j is another node that interacts with i . We also define the truncated history $\mathcal{H}_i^{(m)}(t)$, which only keeps the most recent m events involving node i . The temporal interaction graph framework allows for modeling fine-grained, asynchronous interaction dynamics at event level.

3.3. Problem definition

Given a temporal interaction graph $\mathcal{G} = \{(u_k, v_k, t_k, e_k)\}_{k=1}^M$ and node features $\{\mathbf{x}_i(t)\}_{i \in \mathcal{V}}$, the goal of temporal representation learning is to

generate time-aware embeddings $\mathbf{z}_i(t) \in \mathbb{R}^d$ that encode both structural and temporal dependencies.

The effectiveness of the learned representations is typically evaluated on the following downstream tasks:

- **Dynamic Link Prediction:** Given a quadruple (i, j, t, e_{ij}) and historical interactions before time t , predict whether an interaction between i and j will occur at time t .
- **Dynamic Node Classification:** Given a node i , its historical interactions $\mathcal{H}_i(t)$, and its embedding $\mathbf{z}_i(t)$, predict its class label $\hat{y}_i(t)$ at time t .

These tasks require learning expressive temporal node representations that generalize over evolving graph structures and continuous-time dynamics.

4. Methodology

4.1. Model overview

We tackle two canonical tasks on a temporal interaction graph—link prediction and node classification. Formally, a temporal interaction graph is a sequence $\mathcal{G} = \{EV_k = (u_k, v_k, t_k, e_k)\}_{k=1}^M$ where each time-stamped event connects a source node u_k and a target node v_k at time t_k and may carry an edge feature vector e_k . For the convenience of writing, in the following text, we use i and j to represent a source node and a target node, respectively, similar to u_k and v_k in the definition of TIG in the previous text. We also assume that node $i, j \in \mathcal{V}$. At any query time t we require a low-dimensional node embedding $\mathbf{z}_i(t) \in \mathbb{R}^d$ that is (1) up-to-date, (2) history-aware, and (3) cheap to compute.

State-of-the-art memory-based dynamic graph models such as TGN (Rossi et al., 2020) and TGAT (Xu et al., 2020) update a node's internal memory $s_i(t)$ only when the node participates in an event. This event-driven paradigm implies that the memory is maintained as a discrete-time, piecewise-constant process: between two events, the memory remains frozen and does not evolve in any way.

Such a design prevents the model from obtaining a node's memory at an arbitrary continuous time and ignores natural temporal dynamics—e.g., preferences that decay, drift, or strengthen even when no interaction occurs. As verified in our empirical analysis (Section 4.3), this discrete memory update mechanism results in staleness, which significantly limits the model's ability to capture long-term evolution.

To overcome this issue we design a dual-path framework (Fig. 1) that decouples discrete and continuous dynamics.

To better illustrate the benefit of continuous evolution, consider the three users in Fig. 1: Alice (blue), Bob (green), and Carl (purple). They represent three different customer groups in an e-commerce scenario. On the right side, the three triangles correspond to three types of products: an orange triangle representing “tissue paper” (a frequently purchased daily-use product), a yellow triangle representing “mosquito repellent spray” (an occasional seasonal product), and a gray triangle representing “artisan coffee beans” (a niche product).

Consider Carl (purple). He frequently buys the orange daily-use product (tissue paper), and his last interaction with it occurred at time t_{i-1} . When we later predict whether Carl will interact with some product at time t_{i+1} , the orange product naturally acts as the positive sample, while the yellow seasonal product—mosquito repellent spray—may serve as a negative candidate. During the idle interval (t_{i-1}, t_{i+1}) , the Continuous Evolution (CE) module evolves the memories of all involved nodes: Carl, the positive orange item, and the negative yellow item.

Under this evolution, the memory trajectories are not arbitrary: the Continuous Evolution (CE) module contains learnable parameters that govern how each memory drifts during the idle interval. This allows the model to adjust the direction and magnitude of evolution based on its training objective. In Carl's case, the CE learns a trajectory that gradually moves $s_{Carl}(\tau)$ closer to $s_{orange}(\tau)$ over (t_{i-1}, t_{i+1}) , reflecting Carl's

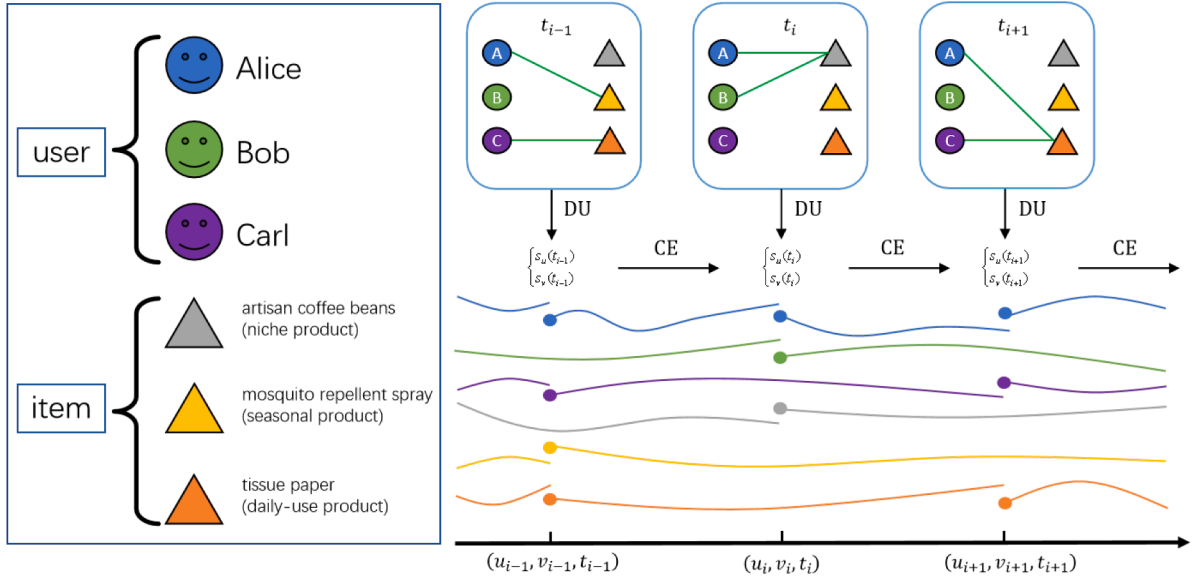


Fig. 1. Event-evolution workflow of our model. At each interaction time t_i (blue squares) the Discrete Update (DU) module consumes the latest event (u_i, v_i, t_i) and refreshes the memories of the involved nodes, yielding up-to-date embeddings $\{s_u(t_i), z_v(t_i)\}$. During the idle interval (t_i, t_{i+1}) , these embeddings are passed through the Continuous Evolution (CE) module, which evolves them smoothly until the next event arrives. This dual-path design captures both instantaneous interaction effects and gradual temporal drift, providing temporally consistent embeddings for downstream prediction.

sustained interest in this daily-use item. At the same time, the CE dynamics push $s_{Carl}(\tau)$ away from $s_{yellow}(\tau)$, because the yellow item—mosquito repellent spray—is repeatedly observed as a negative candidate during training. This controlled drift cannot be captured by purely event-driven models in which all memories remain frozen between interactions.

A similar phenomenon occurs for Alice (blue), but with a different behavioural pattern: Alice does not have strong long-term affinity for any particular item. She may purchase the yellow item (mosquito repellent spray), then switch to the gray item (artisan coffee beans), and later buy the orange item (tissue paper). For such users, the CE module learns an opposite pattern: it does not pull Alice’s memory toward any specific item, but instead encourages her memory to drift away from all previously contacted items after her interaction. This drift reflects a user group whose preferences change rapidly and are not anchored by any particular product category.

As a result, after continuous evolution, Carl’s embedding becomes closer to the orange item and farther from the yellow one, yielding a higher predicted link probability for the orange–Carl pair. In contrast, Alice’s embedding moves away from all three items unless a new event occurs, correctly modelling her unstable preference dynamics. This behaviour contrasts sharply with purely event-driven models where all memories remain frozen between events, preventing such beneficial temporal separation from emerging.

Discrete Update Module (Section 4.2). We inherit the memory-based pipeline of TGN—message generation → message aggregation → state updater. Whenever an event (i, j, t, e_{ij}) arrives, we update the memories $s_i(t)$ and $s_j(t)$ with the event features e_{ij} and the current states of their neighbours. This path excels at capturing instantaneous interaction effects.

Continuous Evolution Module (Section 4.3). Since link prediction at time t^* depends on the relative positions of node embeddings at that time, evolving memories within (t, t^*) is crucial for shaping meaningful distances between positive and negative candidates. In practical prediction scenarios, to predict whether nodes u and v will connect at time t^* , we can only use information from times before t . This means the node’s memory is updated to the latest time point located before t^* . Immediately after the discrete update, each node enters an idle interval (t, t^*) until its next interaction. During this period we evolve its memory ac-

cording to a lightweight time-aware dynamical model:

$$\frac{ds_i(\tau)}{d\tau} = f_\theta(s_i(\tau), \Delta\tau), \quad \tau \in (t, t^*), \quad (4)$$

where f_θ can be an ODE function (Section 4.3.1). Optionally, the latest neighbour embedding can be injected (Eq. (11)) to model short-term preference shifts.

At any query time t^* the node embedding is obtained via a shallow read-out MLP:

$$z_i(t^*) = \text{READOUT}(s_i(t^*)). \quad (5)$$

For link prediction, we feed the concatenation $[z_i(t^*) \parallel z_j(t^*)]$ to an MLP decoder to estimate the probability of interaction between nodes i and j at t^* . For node classification, we attach a linear classifier on top of $z_i(t^*)$.

This dual-path design (1) captures abrupt interaction effects and slow preference drifts simultaneously, (2) keeps computational cost low thanks to closed-form or one-step MLP evolution, and (3) retains compatibility with any memory-based discrete updater. Ablation studies in Section 5.7 verify that continuous evolution consistently improves both AUC and AP over purely event-driven baselines, with negligible overhead.

To summarise the interaction between the discrete memory pipeline and our continuous Natural Evolution Unit (NEU), Fig. 2 illustrates one complete training batch. The diagram highlights-by a black frame-the only component newly introduced by this work; all other blocks are shared with standard memory-based frameworks such as TGN.

4.2. Review of TGN core workflow

Temporal Graph Network (TGN) (Rossi et al., 2020) updates a node’s state in four discrete steps during each interaction event at time t :

Message Function. Computes a message vector for source node i (and target node j) using their previous states $s_i(t^-)$, $s_j(t^-)$, edge features e_{ij} and time encoding:

$$\begin{aligned} \mathbf{m}_i(t) &= \text{MSG}_s(s_i(t^-), s_j(t^-), e_{ij}, \Delta t), \\ \mathbf{m}_j(t) &= \text{MSG}_d(s_j(t^-), s_i(t^-), e_{ij}, \Delta t), \end{aligned} \quad (6)$$

where $\Delta t = t - t_{\text{last}}^{(i)}$ encodes elapsed time since i ’s previous interaction. In the original paper $\text{MSG}(\cdot)$ is chosen as identity (i.e., simple concatenation of the inputs), but learnable MLP variants are equally supported.

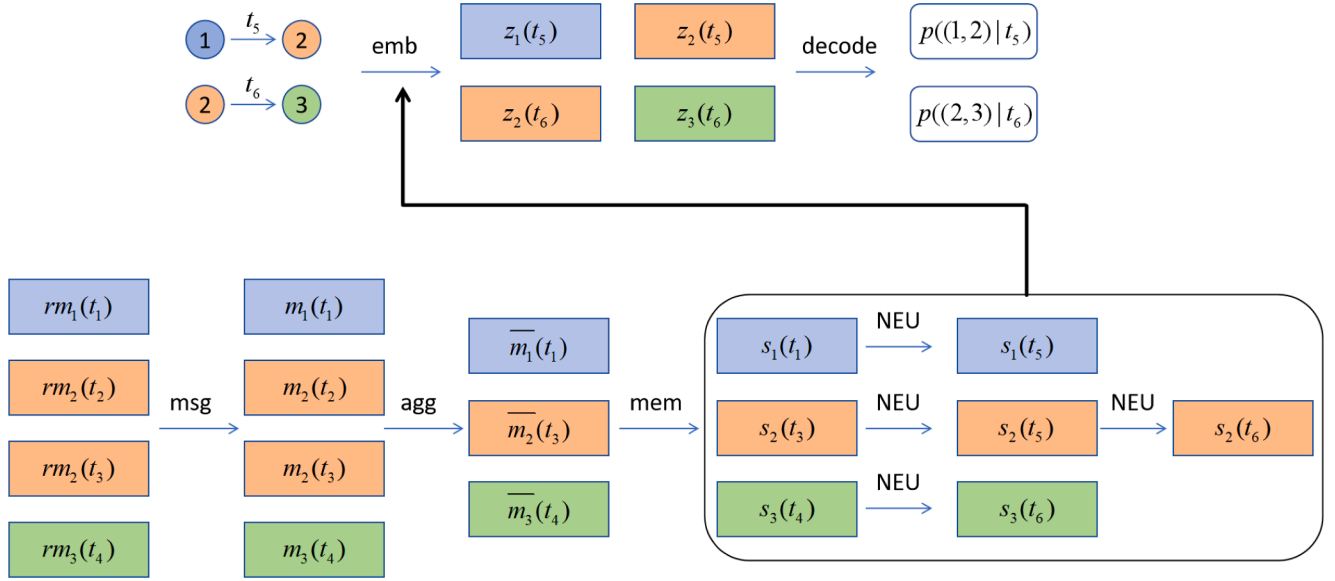


Fig. 2. End-to-end workflow of NEU. **Top:** two events, $(1 \rightarrow 2, t_5)$ and $(2 \rightarrow 3, t_6)$, arrive in the current batch; the model first produces node embeddings $z_i(t)$ and then feeds them to a task-specific decoder to predict link probabilities. **Bottom (left $\rightarrow$ right):** (1) Raw Message Store keeps the latest raw message $rm_i(t)$ for every node; (2) Message Function converts each raw message into a processed message $m_i(t)$; (3) Aggregator merges all messages destined for the same node, yielding $\bar{m}_i(t)$; (4) Memory Updater writes $\bar{m}_i(t)$ into the discrete memory, producing the post-update state $s_i(t)$; (5) **NEU (ours)** integrates the closed-form ODE from t to the event time t^* , obtaining the fresh memory $s_i(t^*)$ and thus preventing staleness; (6) Temporal Graph Attention treats the evolved memories as queries, keys, and values to compute the final embedding $z_i(t^*)$. Except for Step (5), the pipeline is similar to TGN, so continuous memory evolution is achieved with minimal extra cost.

Neighbor Aggregation. Aggregates all messages $\{m_i(t_k)\}$ sent to node i (e.g. via sum or attention) into a single vector:

$$\bar{m}_i(t) = \text{AGG}(\{m_i(t_k)\}), \quad (7)$$

where $t_k \leq t$ are all events involving i . Two non-learnable choices are recommended for efficiency: most-recent (LAST) keeps only the newest message, whereas mean averages all messages.

State Update. Updates the node state via a recurrent or feed-forward unit:

$$s_i(t) = \text{UPDATE}(s_i(t^-), \bar{m}_i(t)), \quad (8)$$

where $\text{UPDATE}(\cdot)$ can be a GRU, LSTM, or any recurrent unit.

Embedding Decoder. Decodes the updated state (optionally with Δt) into the node embedding:

$$z_i(t) = \text{READOUT}(s_i(t), \Delta t). \quad (9)$$

Eqs. (6)–(9) make $s_i(t)$ piecewise constant: it changes only when i participates in an event. As a result, long idle intervals can render memories stale, hindering tasks that depend on up-to-date preferences (see empirical evidence in Section 4.3). Our method in the following sections addresses this issue by inserting a Continuous Evolution path that evolves the memory smoothly between two discrete updates.

4.3. Natural evolution unit (NEU)

Motivation. Existing memory-augmented temporal GNNs such as TGN, DyRep and JODIE refresh a node's memory only at the discrete timestamps where an interaction takes place. Following the symbols set in the previous text, We denote t as the latest interaction time of the source node, t^* as the next interaction time of the source node, which is the time to be predicted. Between two consecutive events (t, t^*) the memory is kept frozen, making the resulting embeddings piece-wise constant. In practice this simplification overlooks three important dynamics. First, user preferences or risk scores often decay intrinsically if no new evidence arrives for a long period. Second, immediately after an interaction the source node is usually biased towards its last neighbour; this short-term drift gradually fades rather than staying constant. Third, nodes

with sparse activity suffer from severely outdated memories, leading to poor predictions on infrequent actors.

Setting of Natural Evolution Unit. To mitigate memory staleness we insert a lightweight continuous-time evolution stage between the discrete memory update (DU) and the embedding read-out. Given the freshly updated memory $s_i(t)$, the elapsed time $\Delta t^* = t^* - t$, and optionally the memory of the most recent neighbour $s_j(t)$, NEU produces a time-consistent memory

$$s_i(t^*) = \text{NEU}(s_i(t), \Delta t^*, s_j(t)), \quad (10)$$

which is then fed to the embedding module. During idle periods, a node's state may either drift toward past stimuli (recent neighbors) or decay away from outdated information. The precise behavior depends on the data. Thus, instead of fixing the decay dynamics, we allow the model to learn the memory evolution pattern directly. To address this, we introduce NEU-ODE (Section 4.3.1) which models exponential decay and neighbour-driven drift via a learnable first-order differential equation with a closed-form solution. In addition, we also propose a more simplified alternative solution. NEU-MLP (Section 4.3.2) offers a lightweight, assumption-free alternative that maps $(s_i(t), \Delta t^*, s_j(t))$ to $s_i(t^*)$ through a two-layer perceptron. Both variants add negligible computational overhead while considerably improving temporal consistency and long-term forecasting accuracy.

Conceptual distinction from event-driven models. Representative temporal graph models such as TGN, TGAT, JODIE, and DyRep largely follow an event-driven paradigm, where node states are updated only when interaction events occur. Between events, memory is typically assumed to remain piecewise constant, and temporal effects are incorporated implicitly through recurrent updates, attention mechanisms, or intensity functions. While effective, this design limits the model's ability to describe fine-grained state transitions over continuous time.

In contrast, NEU decouples memory modeling into event-driven updates and time-driven evolution. By introducing an explicit closed-form continuous-time dynamics, NEU relaxes the piecewise-constant assumption and enables memory states to evolve smoothly and be queried at arbitrary timestamps. This fundamental difference allows NEU to

Table 1

Conceptual comparison between representative event-driven temporal models and NEU.

Aspect	Event-driven Models	NEU (Ours)
Memory Update Trigger	Event-driven only	Event-driven update + Time-driven evolution
State Evolution Between Events	Piecewise constant	Continuous-time closed-form evolution
Temporal Modeling Paradigm	Implicit or discrete-time	Explicit continuous-time dynamics
Memory Query Resolution	Only at event timestamps	Queryable at arbitrary timestamps

capture more precise temporal dynamics while preserving the event-driven backbone of prior methods.

To better clarify the distinctions between NEU and prior temporal graph models, we provide a mechanism-level comparison in Table 1.

4.3.1. Learnable ODE with parametric drift

We propose the following general form for continuous-time evolution:

$$\frac{d s_i(\tau)}{d\tau} = -\lambda s_i(\tau) - \mu s_j(t), \quad \tau \in [t, t^*], \quad (11)$$

where $s_j(t)$ is the updated memory of the most recent neighbor, which we use as a constant in Eq. (11). This design follows the event-driven nature of temporal interaction graph models, where node memories are updated at discrete interaction events by aggregating information from their neighbors. As a result, the influence of the most recent neighbor has already been incorporated into the node's memory at time t . During the subsequent interval without interactions, we model the memory evolution as an autonomous process driven by the node's current state. Under this formulation, the neighbor memory acts as a fixed reference signal that reflects the latest interaction, while the continuous dynamics capture how this influence evolves over time. This allows the model to describe the direction and strength of node's memory drift over time based on previously integrated information.

Moreover, both λ and μ are now treated as learnable parameters—either global scalars or low-rank vectors (e.g., per-head or per-feature), depending on the implementation. Solving Eq. (11) yields:

$$s_i(t^*) = e^{-\lambda \Delta t^*} s_i(t) - \frac{\mu}{\lambda} (1 - e^{-\lambda \Delta t^*}) s_j(t), \quad (12)$$

with $\Delta t^* = t^* - t$.

Setting $\mu > 0$ causes the node state to drift toward the recent neighbor; setting $\mu < 0$ causes the state to diverge from it. The same applies for λ . By learning both, the model can decide—based on the dataset—whether nodes tend to remember (e.g., consistent user interests) or forget (e.g., short-term focus) their past.

For example, a user who just watched a movie may retain interest in similar content (positive drift); another user might shift away after trying a product once (negative drift). By learning both μ and λ , the model adapts naturally to such behaviors without manual assumptions.

After describing the mathematical formula of NEU-ODE, we further illustrate its temporal evolution behavior through previous examples. The details of the black box in Fig. 2 are unfolded as shown in Fig. 3.

As an ablation, we also consider a simpler case where the influence of nearest neighbors is not considered, and only the evolution of the source node itself is applied:

$$\frac{d s_i(\tau)}{d\tau} = -\lambda s_i(\tau). \quad (13)$$

This yields the standard exponential decay:

$$s_i(t^*) = e^{-\lambda \Delta t^*} s_i(t). \quad (14)$$

This variant suits applications where outdated memory should fade without external influence, such as sensor health or passive monitoring systems.

4.3.2. MLP-based memory evolution (No ODE)

While ODEs offer interpretability and continuity, they are not always necessary. We propose a lightweight alternative using a simple MLP:

$$s_i(t^*) = \text{MLP}([s_i(t) \parallel \Delta t^*]). \quad (15)$$

Here, $\text{MLP} : \mathbb{R}^{d+1} \rightarrow \mathbb{R}^d$ learns to evolve the memory directly from the time interval Δt^* and the old state. This design: (1) avoids numerical solvers and ODE libraries; (2) enables end-to-end training on GPU with batched tensors; (3) adapts freely to any empirical drift patterns, without assuming exponential decay or neighbor influence.

We treat this as a non-parametric control experiment to validate whether continuous dynamics are truly necessary, or if temporal evolution can be learned entirely from data via standard neural modules.

The revised full pipeline is:

1. **Discrete Update** at t using TGN's message passing;
2. **Memory Evolution** (our proposed NEU) from $s_i(t)$ to $s_i(t^*)$ via: Eq. (11) (A learnable ODE, as shown in Fig. 3), or Eq. (15) (MLP).
3. **Embedding Readout**: $z_i(t^*) = \text{READOUT}(s_i(t^*))$.

Each variant reflects a different inductive bias. Experiments in Section 5.7 compare their effectiveness across domains.

4.4. Fixed time-encoding function

Time encoding is a fundamental component of temporal GNNs because all neighbor aggregation and message passing depend on the relative time difference between events. Our original method adopts the commonly-used trainable sinusoidal encoder from TGAT (Xu et al., 2020). However, when we applied NEU to long-range temporal datasets, we observed unstable gradients and irregular loss curves. This motivated us to revisit the design of time encoding.

Many previous works represent a timestamp t by a trainable sinusoidal projection:

$$\Phi_{\text{train}}(t) = \cos(t \mathbf{w} + \mathbf{b}), \quad \frac{\partial \Phi_{\text{train}}}{\partial \mathbf{w}} = t \sin(t \mathbf{w} + \mathbf{b}), \quad (16)$$

where $\mathbf{w}, \mathbf{b} \in \mathbb{R}^d$ are learnable parameters. Because the gradient in (16) is proportional to t , large timestamps amplify the magnitude of gradient, leading to exploding gradients, an erratic optimization landscape, and degraded performance.

Following the insight of GraphMixer (Cong et al., 2023), we adopt a fixed positional encoding that decouples the gradient from t :

$$\Phi(t) = \cos(t \boldsymbol{\omega}), \quad \boldsymbol{\omega} = \left[\alpha^{-\frac{0}{\beta}}, \alpha^{-\frac{1}{\beta}}, \dots, \alpha^{-\frac{d-1}{\beta}} \right]^\top, \quad (17)$$

where $\alpha > 1$ controls the minimum wavelength, $\beta > 0$ determines the rate of exponential decay, and d is the encoding dimension. Throughout all experiments we simply set $d = 100$ and $\alpha = \beta = \sqrt{d}$, which satisfy $t_{\max} \alpha^{-(d-1)/\beta} \rightarrow 0$ and can distinguish any timestamp $t \in [0, t_{\max}]$.

Encoding (17) has two desirable features: (1) For $|t_1 - t_2| \ll t_{\max}$ we have $\|\Phi(t_1) - \Phi(t_2)\|_2$ approaching 0, so close timestamps yield similar encodings, facilitating smooth temporal interpolation. (2) Larger timestamps require longer prefixes of $\Phi(t)$ to converge to 1, implicitly encoding chronological order.

We replace the original trainable time encoder of NEU by Eq. (17). Since $\boldsymbol{\omega}$ is fixed, there are no additional parameters. We show that the fixed encoding (1) eliminates gradient explosion, (2) produces a smoother loss landscape, and (3) consistently improves AUC/AP on four benchmark datasets (Table 6).

While more sophisticated encodings (e.g., learned mixtures of periodic bases (Tancik et al., 2020)) could be explored, our experiments indicate that the simple form (17) already provides a stable and expressive temporal signal when combined with the proposed Natural Evolution Unit.

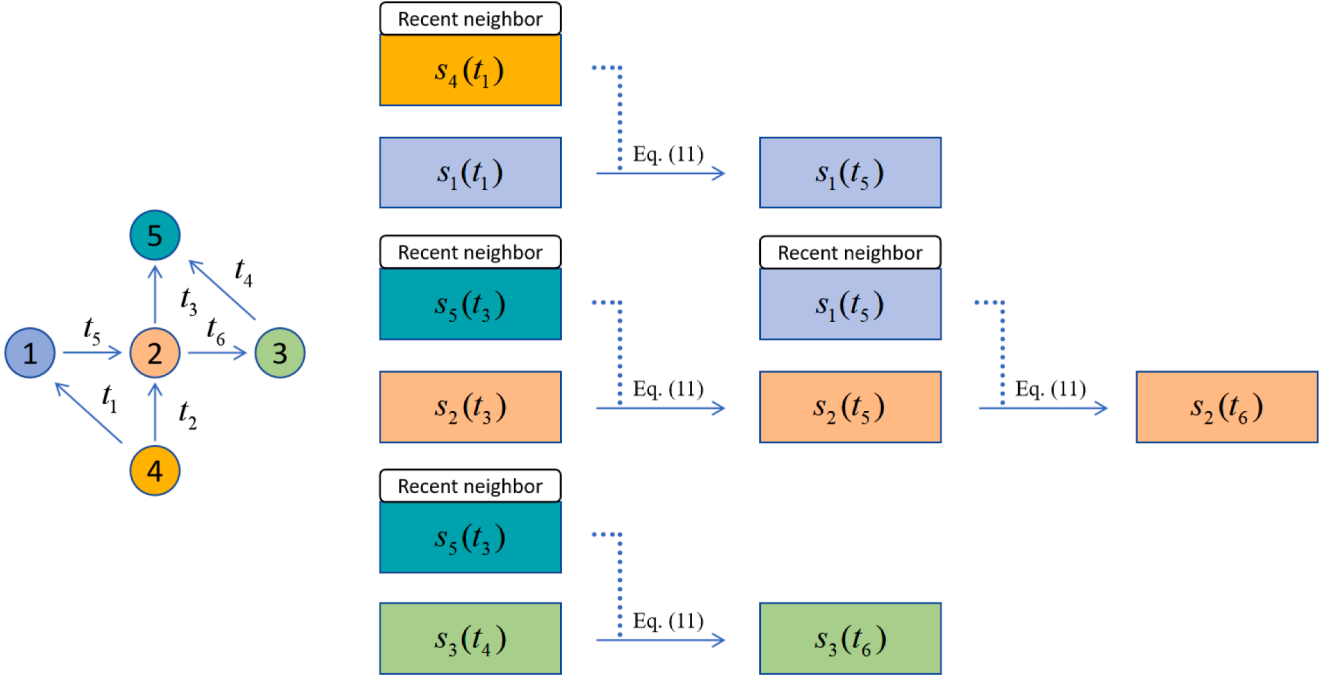


Fig. 3. Illustration of the NEU-ODE mechanism. In each batch, the latest memory of a target node evolves from its previous memory (e.g., $s_1(t_1)$) toward its current state (e.g., $s_1(t_5)$), influenced jointly by its own historical memory and the recent memories of its temporal neighbors (e.g., $s_4(t_1)$). The learnable ODE governs this evolution, enabling the model to capture smooth and realistic state transitions between asynchronous interaction events.

4.5. Attention embedding module

We adopt the temporal graph-attention layer first introduced in TGAT (Xu et al., 2020) to convert evolved memory $s_i(t^*)$ into a task-ready embedding. The meaning of t^* remains the same as before, representing the time to be predicted after NEU. In the following process, our goal is to calculate the embedding of t^* to connect downstream tasks, which is different from t in previous models such as TGN. In contrast to TGAT/TGN, we replace their learnable time encoder with the **Fixed Time-Encoding Function** $\phi(\Delta t)$ proposed in Section 4.4. This deterministic encoding speeds up convergence and reduces over-fitting, while our NEU has already injected most of the temporal signal into $s_i(t^*)$.

At each attention layer l we define the query as the previous representation of node i ,

$$\mathbf{q}_i^{(l)}(t^*) = h_i^{(l-1)}(t^*) \in \mathbb{R}^d, \quad (18)$$

and build key / value matrices by concatenating (1) the most recent memory of each neighbour $j \in \mathcal{N}_i(t)$, (2) the edge feature e_{ij} , and (3) the fixed time code $\phi(\Delta t)$ with $\Delta t = t - t^-$ is the timestamp of the last interaction (i, j) observed before t :

$$\mathbf{K}_i^{(l)}(t^*) = \mathbf{V}_i^{(l)}(t^*) = \left[\left[h_j^{(l-1)}(t^*) \parallel e_{ij} \parallel \phi(\Delta t) \right] \right]_{j \in \mathcal{N}_i(t^*)} \in \mathbb{R}^{N \times d}, \quad (19)$$

where $\parallel$ denotes vector concatenation. We then apply standard multi-head attention (Vaswani et al., 2017):

$$\tilde{h}_i^{(l)}(t^*) = \text{MultiHeadAttention}^{(l)}(\mathbf{q}_i^{(l)}(t^*), \mathbf{K}_i^{(l)}(t^*), \mathbf{V}_i^{(l)}(t^*)) \in \mathbb{R}^d. \quad (20)$$

Finally, we fuse the aggregated neighbor information with the node's own state:

$$z_i(t^*) = h_i^{(l)}(t^*) = W_o^{(l)}[s_i(t^*) \parallel \tilde{h}_i^{(l)}(t^*)] \in \mathbb{R}^d, \quad (21)$$

where $W_o^{(l)} \in \mathbb{R}^{d \times 2d}$ is a learnable output projection. After L stacked layers, the top-layer output $z_i(t^*)$ is used as the final embedding.

By eliminating the learned time encoder, the attention block focuses purely on structural interactions and the already time-aware memories $s_i(t)$. Empirically (Section 5.7) this yields faster training, better generalization, and lower parameter count compared with TGAT/TGN. For graphs without edge attributes we simply drop e_{ij} from the concatenation, leaving the formulation intact.

4.6. Decoder

Following standard practice in temporal interaction modeling (Rossi et al., 2020), we use a two-layer MLP decoder to compute the probability of future interactions. Given the embeddings of node i and node j at time t^* , denoted by $z_i(t^*)$ and $z_j(t^*)$, the decoder predicts the likelihood of an interaction event (i, j, t^*) as follows:

$$\hat{p}_{ij}(t^*) = \sigma(\text{MLP}([z_i(t^*) \parallel z_j(t^*)])), \quad (22)$$

where $\sigma(\cdot)$ is the sigmoid activation and $[\cdot \parallel \cdot]$ denotes concatenation. To train the model, we adopt the binary cross-entropy loss:

$$\mathcal{L} = - \sum_{(i,j,t^*) \in \mathcal{E}} [\log \hat{p}_{ij}(t^*) + \log(1 - \hat{p}_{ik}(t^*))], \quad (23)$$

where k is a negative sample randomly drawn from the set of nodes and $\hat{p}_{ik}(t^*)$ is computed in the same way as $\hat{p}_{ij}(t^*)$. This decoder design provides a flexible and self-supervised learning signal for both temporal link prediction and node classification tasks.

For the node-level task, we project the time-dependent embedding of node i through a *softmax* linear classifier

$$\hat{\mathbf{y}}_i(t^*) = \text{softmax}(\mathbf{W}_{\text{cls}} z_i(t^*) + \mathbf{b}_{\text{cls}}), \quad (24)$$

with trainable parameters $\mathbf{W}_{\text{cls}} \in \mathbb{R}^{C \times d}$ and $\mathbf{b}_{\text{cls}} \in \mathbb{R}^C$, where d is the embedding dimension and C the number of classes. Model training minimises the standard cross-entropy between the predicted distribution $\hat{\mathbf{y}}_i(t^*)$ and the ground-truth one-hot label $\mathbf{y}_i$:

$$\mathcal{L}_{\text{cls}} = - \sum_i \sum_{c=1}^C y_{ic} \log \hat{y}_{ic}(t^*). \quad (25)$$

Table 2
Statistics of datasets used in our experiments.

Dataset	# Nodes	# Edges	# Edge Features	# Labels
Wikipedia	9227	157,474	172	Available
Reddit	10,984	672,447	172	Available
MOOC	7144	411,749	4	Available
LastFM	1980	1,293,103	0	–
MovieLens-100K	2625	100,000	0	–

4.7. Training strategy

Our training procedure follows the general framework of memory-based temporal GNNs such as TGN (Rossi et al., 2020), where memory states are updated in a causal and non-leaking fashion. At each training step, a batch of time-ordered interaction events $\{(i, j, t, e_{ij})\}$ is processed.

To prevent information leakage, the memory states used for embedding computation at time t are obtained from the latest interactions prior to t (i.e., those from previous batches). We use the raw message store to cache the last interaction messages for each node and update memory accordingly before processing the current batch.

However, unlike TGN, we posit that node memories may evolve even during periods of inactivity. To model this, we insert a continuous state evolution step before the discrete message-passing update. Specifically, we evolve each node's memory state $s_i(t)$ forward in time using our ODE-based or MLP-based evolution module to obtain $s_i(t^*)$, which reflects the natural evolution of the node's representation over the interval $[t, t^*]$:

- If using an ODE evolution module, we solve an interpretable differential equation (see Section 4.3).
- If using an MLP-based evolution module, we apply a learned transformation of $(s_i(t), \Delta t)$ to compute $s_i(t^*)$.

This updated state $s_i(t^*)$ is then used as input to the temporal embedding module (e.g., TGAT), which aggregates neighbor information to produce the final embedding $z_i(t^*)$ for decoding. The overall computation process is illustrated in Fig. 2.

Importantly, all interactions in a batch use the same memory state, evolved from the outputs of the previous batch. This avoids introducing information from future events within the same batch, ensuring causal consistency. Compared with TGN, our model introduces only a small computational overhead but significantly improves the temporal expressiveness of node embeddings.

5. Experiments

5.1. Datasets and baselines

To evaluate the effectiveness of our proposed method, we conduct experiments on five benchmark datasets: *Wikipedia*, *Reddit*, *MOOC*, *LastFM*,¹ and *MovieLens-100K*.² These datasets cover diverse application domains, including online discussions (Wikipedia, Reddit), online learning (MOOC), music streaming (LastFM), and movie ratings (MovieLens-100K). Following prior work (Rossi et al., 2020; Xu et al., 2020), edges in these datasets represent timestamped interactions between nodes. For datasets such as Wikipedia, Reddit, and LastFM, dynamic binary labels are provided to indicate the state of user nodes over time. Node and edge attributes are not available in most datasets, except for Wikipedia, Reddit and MOOC which contain edge features. The statistics of the datasets are summarized in Table 2.

For all datasets, we adopt the standard temporal train / validation / test split by chronologically dividing the edge stream into 70% training,

15% validation, and 15% test sets, ensuring no future information leaks during training.

We compare our method against seven strong baselines. TGAT (Xu et al., 2020) serves as a representative memory-less dynamic graph model that leverages temporal attention. JODIE (Kumar et al., 2019), DyRep (Trivedi et al., 2019), and TGN (Rossi et al., 2020) are memory-based models that maintain and update node memories over time. After TGN, there are also some temporal interaction graph models. We selected three representative models for comparison: GraphMixer (Cong et al., 2023) is an MLP-based temporal graph model that replaces attention and message passing with token-mixing and channel-mixing operations. DyGFormer (Yu et al., 2023) is a transformer-based architecture for dynamic graph learning. FreeDyG Tian et al. (2024) is a frequency enhanced continuous-time dynamic graph model. These methods are the most relevant comparisons, as our model also adopts a memory mechanism but distinguishes itself by enabling continuous memory evolution between interactions. Among them, TGN is considered the state-of-the-art due to its flexible design and superior performance across dynamic graph tasks.

5.2. Evaluation tasks and metrics

We evaluate all models on the dynamic link prediction task under both transductive and inductive settings. In the transductive setting, models predict future interactions among nodes that appeared during training. In contrast, the inductive setting requires generalizing to previously unseen nodes during inference, which better reflects real-world deployment scenarios.

To comprehensively assess performance, we report both Average Precision (AP) and Area Under the ROC Curve (AUC) as evaluation metrics, by sampling an equal number of negative examples for each positive interaction, following prior protocols (Rossi et al., 2020; Xu et al., 2020). These metrics jointly reflect the models' ranking quality and overall discriminative ability for temporal link prediction.

5.3. Hyperparameters and environment

Hyperparameters. We use a two-layer MLP as the classifier in downstream tasks. For all models, we adopt the Adam optimizer with a learning rate of 0.0001 and set the batch size to 200. The dropout rate is set to 0.1, and the number of attention heads is 2. We use one GNN layer with 100-dimensional node embeddings and 100-dimensional time encodings. The memory and message dimensions are set to 172 and 100, respectively, consistent with the edge feature dimension in Wikipedia and Reddit. For all datasets, we sample 10 temporal neighbors per node, train for up to 50 epochs, and apply early stopping with a patience of 5. Gradients are backpropagated every batch. All experiments are conducted with 1 run per setting. In our main experiments, the hyperparameters were set according to the recommended settings in TGN (Rossi et al., 2020). To ensure fair comparisons, we keep the hyperparameters fixed across all datasets.

Environment. All experiments are conducted on a Linux server with the following configuration: Ubuntu 20.04.6 LTS, Python 3.9.20, PyTorch 1.12.1, CUDA 11.4, NVIDIA Driver 470.256.02, r9 CPU, and NVIDIA RTX 3090 GPU.

¹ <http://snap.stanford.edu/jodie/>

² <https://grouplens.org/datasets/movielens/>

Table 3AUC (%) for future edge prediction in transductive and inductive settings. Best results are in **bold**, and second best are underlined.

Setting	Dataset	JODIE	TGAT	DyRep	GraphMixer	DyGFormer	FreeDyG	TGN	NEU-ODE	NEU-MLP
Transductive	Wikipedia	91.70	90.04	90.43	95.46	98.38	<u>98.83</u>	97.85	98.84	98.10
	Reddit	97.53	98.13	98.24	97.24	<u>99.00</u>	98.95	98.67	99.72	98.79
	MOOC	84.32	73.65	82.84	83.59	<u>86.38</u>	88.52	91.49	97.39	<u>91.65</u>
	LastFM	70.85	51.07	71.56	74.34	89.39	<u>89.53</u>	79.50	93.62	84.37
	MovieLens-100K	80.04	79.57	80.11	80.95	80.20	82.89	86.12	99.16	<u>86.51</u>
Inductive	Wikipedia	89.84	85.91	87.99	94.08	<u>97.68</u>	98.14	96.64	97.46	97.01
	Reddit	95.15	96.52	96.47	94.79	<u>98.48</u>	98.43	97.60	98.58	97.72
	MOOC	83.15	72.42	82.50	82.16	<u>85.49</u>	87.76	89.76	96.08	<u>91.03</u>
	LastFM	82.60	52.39	84.66	81.16	<u>91.58</u>	92.11	85.07	90.30	89.02
	MovieLens-100K	76.16	76.24	75.99	76.98	<u>75.65</u>	79.47	<u>84.05</u>	99.05	84.01

Table 4Average Precision (%) for future edge prediction in transductive and inductive settings. Best results are in **bold**, and second best are underlined.

Setting	Dataset	JODIE	TGAT	DyRep	GraphMixer	DyGFormer	FreeDyG	TGN	NEU-ODE	NEU-MLP
Transductive	Wikipedia	91.23	90.87	91.67	96.00	98.62	<u>98.87</u>	98.07	98.88	98.29
	Reddit	96.73	98.26	98.28	97.37	<u>99.12</u>	<u>99.01</u>	98.71	99.67	98.82
	MOOC	80.13	72.11	79.87	82.36	<u>86.27</u>	88.40	89.91	96.61	<u>90.09</u>
	LastFM	70.49	53.80	72.20	76.22	<u>90.76</u>	90.67	78.84	93.58	84.26
	MovieLens-100K	77.26	77.18	77.23	78.28	74.60	80.39	84.43	99.12	<u>85.00</u>
Inductive	Wikipedia	90.33	87.04	89.33	94.76	<u>97.93</u>	98.23	96.94	97.57	97.23
	Reddit	94.16	96.73	96.48	95.09	98.69	<u>98.56</u>	97.65	98.51	97.83
	MOOC	79.15	70.78	79.57	80.81	85.12	87.67	88.45	95.26	<u>89.78</u>
	LastFM	83.00	54.84	85.24	82.69	<u>92.65</u>	92.93	84.70	91.31	89.11
	MovieLens-100K	73.61	74.54	73.96	74.73	70.08	77.34	<u>82.60</u>	98.93	82.46

Table 5Node classification accuracy (%) on datasets with node labels. Best results are in **bold**.

Dataset	JODIE	DyRep	TGN	NEU-ODE (Ours)
Wikipedia	86.43	83.38	87.42	87.66
Reddit	63.51	62.72	65.98	68.98
MOOC	64.56	67.18	65.79	66.19

5.4. Temporal link prediction

We evaluate our model on the task of future link prediction under two standard settings: **Transductive**, where the model predicts interactions between nodes that appeared during training, and **Inductive**, where predictions are made for previously unseen nodes. For each timestamped interaction, the model is asked to estimate the probability that an edge occurs between a given source and destination node. Following standard protocols, we report both AUC and Average Precision (AP) as evaluation metrics.

We compare our approach against a comprehensive set of strong baselines, including classical memory-based models (JODIE, DyRep, TGN), memory-less architectures (TGAT), as well as several recent state-of-the-art scalable temporal models, namely GraphMixer, DyGFormer, and FreeDyG. These baselines represent the latest advances in temporal graph representation learning and significantly strengthen the competitiveness of our evaluation.

As reported in [Tables 3 and 4](#), our proposed NEU-ODE model consistently achieves the best or second best performance across almost all datasets and both evaluation settings. In particular, NEU-ODE establishes new state-of-the-art results on challenging datasets such as MOOC and MovieLens-100K, where the improvements over the strongest baselines are especially pronounced. These gains demonstrate the effectiveness of explicitly modeling continuous-time memory evolution for capturing complex temporal dynamics.

Compared with advanced transformer-based models such as DyGFormer and efficient mixing-based approaches like GraphMixer and FreeDyG, NEU-ODE consistently yields superior results, highlighting

that continuous-time dynamical modeling provides complementary advantages beyond architectural scaling alone. Notably, even against TGN—the strongest representative among memory-based methods—NEU-ODE delivers clear and consistent improvements under both transductive and inductive settings.

We further report results for a simplified variant, NEU-MLP, which replaces the ODE-based memory evolution with MLP. Although NEU-MLP still has a certain gap compared to NEU-ODE, it can almost always outperform methods without memory evolution such as TGN, especially on datasets with long-term temporal dependencies. This performance gap underscores the importance of continuous-time evolution in learning expressive and temporally coherent node representations.

Overall, these results validate the effectiveness and robustness of our proposed temporal modeling approach across diverse domains and dynamic graph patterns.

Beyond final metrics, we analyse the optimisation behaviour of both models. [Figs. 4 and 5](#) report (a) training loss, (b) validation AUC/AP, and (c) seconds per epoch.

For the Wikipedia dataset (small-to-medium scale), NEU-ODE reaches a stable minimum after only 10 epochs, while TGN still descends slowly ([Fig. 4a](#)). The gap on validation metrics emerges early ([Fig. 4b](#)) and remains roughly +1.5%AUC / +1.3%AP throughout training. Runtime overhead is negligible: 5% longer per epoch ([Fig. 4c](#)).

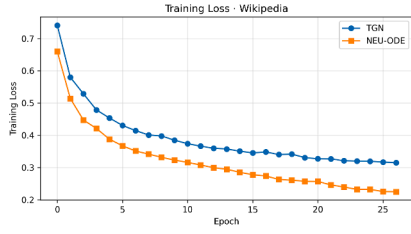
Reddit is over $20 \times$ larger than Wikipedia in number of events, hence the absolute epoch time is much higher for both models (three minutes vs. 25 s). Nevertheless, [Fig. 5](#) shows a similar pattern: NEU-ODE consistently achieves lower loss and higher AUC/AP, converging in 15 epochs, while TGN plateaus later and at a lower score. Importantly, the per-epoch cost of NEU-ODE is only 4% above TGN despite the additional continuous update, confirming that our design scales gracefully.

Across two datasets of very different scales, NEU-ODE (1) converges faster, (2) produces superior validation metrics, and (3) introduces only marginal wall-clock overhead. These observations strengthen the claim that continuous memory evolution brings consistent gains without sacrificing efficiency.

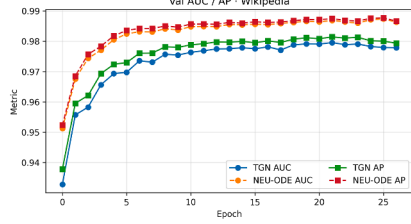
Table 6

Ablation study on temporal link prediction. We report AUC / AP (in percentage) for both **transductive** and **inductive** settings. Best results per row are in **bold**, and second best are underlined.

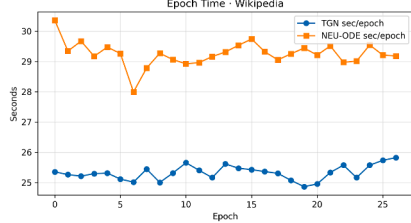
Setting	Dataset	Full Model	w/o Neighbor Evolution	w/o NEU	w/o Fixed Time Encoding	TGN
Transductive	Wikipedia	98.63 / 98.68	98.10 / 98.29	98.13 / 98.29	<u>98.22 / 98.35</u>	97.85 / 98.07
	Reddit	99.72 / 99.67	98.78 / 98.81	98.77 / 98.80	<u>98.74 / 98.76</u>	98.67 / 98.71
	MOOC	97.39 / 96.61	92.24 / 90.47	92.31 / 90.77	<u>97.20 / 96.43</u>	91.49 / 89.91
	LastFM	93.62 / 93.58	83.99 / 83.78	84.91 / 84.79	<u>85.50 / 85.82</u>	79.50 / 78.84
	MovieLens-100K	99.16 / 99.12	86.59 / 85.00	86.98 / 85.41	<u>97.80 / 97.25</u>	86.12 / 84.43
Inductive	Wikipedia	97.30 / 97.48	<u>97.15 / 97.41</u>	97.02 / 97.28	97.05 / 97.31	96.64 / 96.94
	Reddit	98.58 / 98.51	<u>97.93 / 98.02</u>	97.69 / 97.77	97.64 / 97.70	97.60 / 97.65
	MOOC	96.08 / 95.26	92.65 / 91.07	92.21 / 90.94	<u>95.69 / 94.89</u>	89.76 / 88.45
	LastFM	90.30 / 91.31	88.08 / 87.93	88.42 / 88.57	<u>89.14 / 89.49</u>	85.07 / 84.70
	MovieLens-100K	99.05 / 98.93	84.06 / 82.52	84.55 / 83.00	<u>97.97 / 97.58</u>	84.05 / 82.60



(a) Training loss



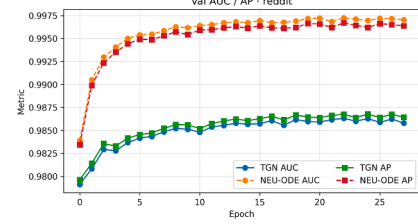
(b) Validation AUC/AP



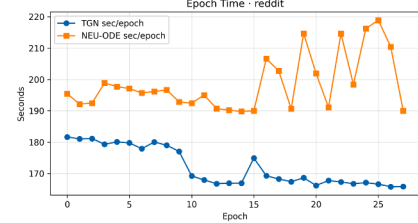
(c) Epoch time

Fig. 4. Training curves on *Wikipedia*.

(a) Training loss



(b) Validation AUC/AP



(c) Epoch time

Fig. 5. Training curves on *Reddit*.

5.5. Node classification

To evaluate the effectiveness of the learned temporal node representations, we conduct a dynamic node classification task on datasets with node-level labels, including *Wikipedia*, *Reddit*, and *MOOC*. Following previous work, we train a two-layer MLP classifier using node embeddings $h_i(t^-)$ generated immediately before the classification time. The goal is to predict whether a node undergoes a state change, such as being banned or dropping out.

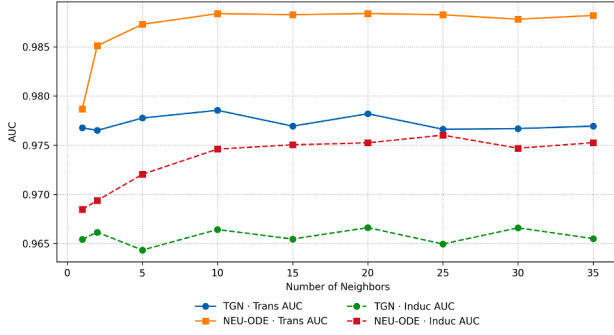
We compare our proposed methods with TGN, the most competitive memory-based baseline. As shown in [Table 5](#), our model achieves better accuracy in both *Wikipedia* and *Reddit* datasets. In particular, the ODE-based memory evolution mechanism significantly enhances node representation quality by modeling the continuous temporal dynamics between interaction events. These results validate the superiority of

modeling natural memory evolution in sparse-event environments for downstream classification tasks.

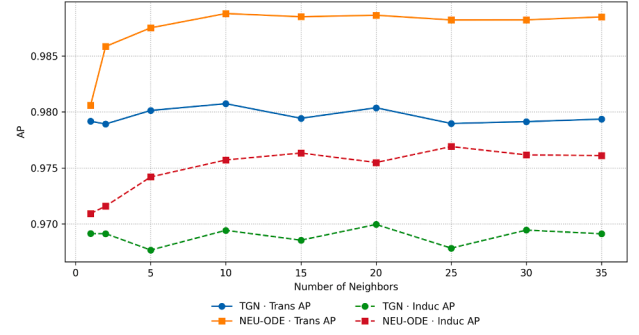
5.6. Computational complexity analysis

We analyze the additional computational cost introduced by the proposed continuous-time memory evolution and compare it with the original TGN framework.

For each interaction event, the dominant computational cost of TGN arises from two components: (1) neighbor sampling and aggregation, and (2) memory update (e.g., GRU/MLP-based update). Given embedding dimension d and sampled neighborhood size K , the aggregation step requires $\mathcal{O}(Kd)$ operations, while the memory update typically involves matrix-vector multiplications with complexity up to $\mathcal{O}(d^2)$. These components dominate the overall time complexity of the model.



(a) Effect of the number of neighbors on AUC.



(b) Effect of the number of neighbors on AP.

Fig. 6. Effect of the number of neighbors on future link prediction performance (Wikipedia dataset).

In contrast, the proposed continuous-time evolution module operates on each node memory using a linear ODE with a closed-form solution. For a memory vector $s_i \in \mathbb{R}^d$, the evolution from time t to t^* involves a constant number of element-wise operations, including one scalar exponential computation and vector-scalar multiplications. Therefore, the additional computational cost per update is $\mathcal{O}(d)$, without any iterative time discretization or numerical solver. Since $\mathcal{O}(d)$ is strictly lower-order compared to $\mathcal{O}(d^2)$ and does not scale with the neighborhood size K , the proposed module does not change the asymptotic complexity of the overall framework. In practice, its overhead is negligible relative to aggregation and memory update operations.

Moreover, we replace the original trainable time encoding $\phi_{\text{train}}(t)$, which requires gradient computation, with a fixed harmonic encoding $\phi(t)$, where ω is predefined. The trainable encoding introduces additional parameter storage and gradient computation proportional to d , whereas the fixed encoding eliminates parameter updates and corresponding backward-pass costs. Thus, while the closed-form ODE adds $\mathcal{O}(d)$ forward computation, the removal of trainable temporal parameters reduces backward computation and parameter update overhead.

Overall, the total time complexity of our model remains dominated by the original aggregation and memory update steps of TGN, preserving scalability. The additional memory overhead is $\mathcal{O}(1)$ due to only two learnable scalar parameters in the ODE module. Hence, our method enhances temporal expressiveness without increasing the asymptotic computational complexity of the framework.

5.7. Ablation study

To better understand the impact of each design component, we conduct an ablation study on the temporal link prediction task under both transductive and inductive settings. The evaluation is performed on the five datasets mentioned in the previous experiment: *Wikipedia*, *Reddit*, *MOOC*, *LastFM*, and *MovieLens-100K*. We report both AUC and AP metrics.

We consider the following variants of our model:

- **Full Model:** Our complete framework, incorporating the Natural Evolution Unit (NEU) with fixed-time encoding.
- **w/o Neighbor Evolution:** This variant removes the neighbor evolution term in the differential equation of the NEU while keeping the continuous evolution mechanism.
- **w/o NEU:** This variant completely removes the natural evolution unit, updating node memory only at discrete interaction events.
- **w/o Fixed Time Encoding:** This variant replaces our fixed sine-cosine-based time encoding with the learnable positional encoding used in TGAT.
- **TGN:** We include the Temporal Graph Network (TGN) as a strong memory-based baseline for comparison.

From the results summarized in Table 6, we make the following key observations. First, the proposed full model consistently achieves the best performance across all datasets and settings, demonstrating the effectiveness of integrating the Natural Evolution Unit (NEU) with fixed time encoding. Second, removing the neighbor evolution term (w/o Neighbor Evolution) leads to noticeable performance degradation, especially in datasets like *MOOC* and *MovieLens-100K*, highlighting the importance of modeling temporal influence attenuation from neighbors. Third, ablating the entire NEU module (w/o NEU) causes further performance drop in most cases, confirming the necessity of continuous memory evolution. Finally, replacing the fixed-time encoding with learnable positional encoding (w/o Fixed Time Encoding) also results in lower performance, suggesting that our simple but stable time representation helps avoid training instability. Overall, the ablation studies validate each design component of our model and underscore their complementary contributions to temporal link prediction accuracy.

5.8. Parameter study

To further investigate the impact of the number of sampled neighbors, we conduct a series of ablation experiments on the Wikipedia dataset by varying the number of neighbors in $\{1, 2, 5, 10, 15, 20, 25, 30, 35\}$. As shown in Fig. 6, we report AUC and AP results for both transductive and inductive link prediction settings.

Our NEU-ODE model consistently outperforms the baseline TGN across all neighbor configurations, demonstrating strong robustness to hyperparameter variation. The performance of NEU-ODE remains stable and peaks when sampling 15-25 neighbors, which shows its capability to effectively leverage local historical context without being sensitive to over-sampling noise. Notably, the performance gap is more significant in the transductive setting, which indicates that our model benefits more from stable structural patterns in seen nodes.

In the inductive setting, NEU-ODE maintains competitive AUC and AP scores as the number of neighbors increases, while TGN shows more fluctuation. This suggests that our dual mechanism design is more effective in generalizing to unseen node interactions. These observations further validate the importance of adaptive neighbor modeling for robust temporal representation learning.

6. Conclusion

In this paper, we propose a novel memory-based temporal interaction graph neural network that addresses a critical limitation in existing dynamic graph models: the staleness of node representations between interactions. To tackle this, we introduce the Natural Evolution Unit (NEU), a continuous-time module based on ordinary differential equations (ODEs), which enables node memory to evolve smoothly over time even in the absence of events. This design allows our model to better

capture fine-grained temporal dynamics and maintain up-to-date representations across varying interaction frequencies. In addition, we adopt a fixed time encoding scheme that stabilizes training and reduces sensitivity to temporal noise. Extensive experiments on six benchmark datasets demonstrate the superiority of our model in both link prediction and node classification tasks, and the ablation studies confirm the importance of each proposed component.

In future work, we plan to extend the NEU with more expressive ODE parameterizations, explore adaptive mechanisms for time encoding across datasets, and integrate our approach into more scalable graph learning systems for large-scale industrial applications. We believe our work provides a new perspective on modeling temporal continuity in dynamic graphs and paves the way for more robust and interpretable dynamic representation learning.

CRedit authorship contribution statement

Yang Meng: Writing – original draft, Software, Methodology, Conceptualization; **Yuting Liu:** Writing – review & editing, Supervision, Funding acquisition.

Data availability

I have shared a link to my data in the experimental section of the manuscript.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Acknowledgement

This work was supported by the Beijing Natural Science Foundation under Grant L222051.

References

- Battaglia, P.W., Hamrick, J.B., Bapst, V., Sanchez-Gonzalez, A., Zambaldi, V., Malinowski, M., Tacchetti, A., Raposo, D., Santoro, A., Faulkner, R. et al. (2018). Relational inductive biases, deep learning, and graph networks. *arXiv preprint arXiv:1806.01261*.
- Bronstein, M.M., Bruna, J., LeCun, Y., Szlam, A., & Vandergheynst, P. (2017). Geometric deep learning: Going beyond euclidean data. *IEEE Signal Processing Magazine*, 34(4), 18–42.
- Chamberlain, B., Rowbottom, J., Gorinova, M.I., Bronstein, M., Webb, S., & Rossi, E. (2021). Grand: Graph neural diffusion. In *International conference on machine learning* PMLR. (pp. 1407–1418).
- Choi, J., Hong, S., Park, N., & Cho, S.-B. (2023). Gread: Graph neural reaction-diffusion networks. In *Proceedings of the 40th international conference on machine learning (ICML)*. *arXiv preprint arXiv:2211.14208*.
- Cong, W., Zhang, S., Kang, J., Yuan, B., Wu, H., Zhou, X., Tong, H., & Mahdavi, M. (2023). Do we really need complicated model architectures for temporal networks? In *Proceedings of the 37th conference on neural information processing systems (neurIPS)*. GraphMixer. <https://doi.org/10.48550/arXiv.2302.11636>
- Dai, H., Wang, Y., Trivedi, R., & Song, L. (2016). Deep coevolutionary network: Embedding user and item features for recommendation. <https://arxiv.org/abs/1609.03675>.
- Duan, P., Ren, X., & Liu, Y. (2023). Multi-relational dynamic graph representation learning. *Neurocomputing*, 558, 126688. <https://doi.org/10.1016/j.neucom.2023.126688>
- Dunlavy, D.M., Kolda, T.G., & Acar, E. (2011). Temporal link prediction using matrix and tensor factorizations. *ACM Transactions on Knowledge Discovery from Data*, 5(2). <https://doi.org/10.1145/1921632.1921636>
- Eliasof, M., Haber, E., & Treister, E. (2021). PDE-GCN: Novel architectures for graph neural networks motivated by partial differential equations. In *Advances in neural information processing systems* (vol. 34). (pp. 3836–3849).
- Eliasof, M., Haber, E., & Treister, E. (2024). Feature transportation improves graph neural networks. In *Proceedings of the AAAI conference on artificial intelligence (AAAI)*. *arXiv preprint arXiv:2307.16092*.
- Fang, Z., Long, Q., Song, G., & Xie, K. (2021). Spatial-temporal graph ODE networks for traffic flow forecasting. In *Proceedings of the 27th ACM SIGKDD conference on knowledge discovery & data mining (KDD '21)* ACM. (pp. 364–373). <https://doi.org/10.1145/3447548.3467430>
- Gilmer, J., Schoenholz, S.S., Riley, P.F., Vinyals, O., & Dahl, G.E. (2017). Neural message passing for quantum chemistry. In *Proceedings of the 34th international conference on machine learning* PMLR (pp. 1263–1272). (vol. 70).
- Grover, A., & Leskovec, J. (2016). Node2Vec: Scalable feature learning for networks. In *Proceedings of the 22nd ACM SIGKDD international conference on knowledge discovery and data mining* (pp. 855–864).
- Hamilton, W.L., Ying, R., & Leskovec, J. (2017). Inductive representation learning on large graphs. In *Advances in neural information processing systems*. (vol. 30).
- Han, L., Zhang, R., Sun, L., Du, B., Fu, Y., & Zhu, T. (2023). Generic and dynamic graph representation learning for crowd flow modeling. In *Proceedings of the AAAI conference on artificial intelligence* Washington, DC, USA (vol. 37). (pp. 4293–4301).
- Jiang, W., & Luo, J. (2022). Graph neural network for traffic forecasting: A survey. *Expert systems with applications*, 207, 117921.
- Ju, W., Fang, Z., Gu, Y., Liu, Z., Long, Q., Qiao, Z., Qin, Y., Shen, J., Sun, F., Xiao, Z. et al. (2024). A comprehensive survey on deep graph representation learning. *Neural Networks*, 173, 106207.
- Kang, W.-C., & McAuley, J. (2018). Self-attentive sequential recommendation. In *201 IEEE international conference on data mining (ICDM)* IEEE. (pp. 197–206).
- Kazemi, S.M., Goel, R., Jain, K., Kobyzev, I., Sethi, A., Forsyth, P., & Poupart, P. (2020). Representation learning for dynamic graphs: A survey. *Journal of Machine Learning Research*, 21(70), 1–73. <http://jmlr.org/papers/v21/19-447.html>.
- Kipf, T.N., & Welling, M. (2017). Semi-supervised classification with graph convolutional networks. In *International conference on learning representations (ICLR)*.
- Kumar, S., Zhang, X., & Leskovec, J. (2019). Predicting dynamic embedding trajectory in temporal interaction networks. In *Proceedings of the 25th ACM SIGKDD international conference on knowledge discovery & data mining* Association for Computing Machinery. (p. 1269–1278). <https://doi.org/10.1145/3292500.3330895>
- Li, Y., Wang, X., Liu, H., & Shi, C. (2024). A generalized neural diffusion framework on graphs. In M.J.J. Wooldridge, J.G. Dy, & S. Natarajan (Eds.), *Proceedings of the thirty-eighth AAAI conference on artificial intelligence (AAAI)* AAAI Press. Vancouver, Canada: (pp. 8707–8715). <https://doi.org/10.1609/aaai.v38i8.28716>
- Liben-Nowell, D., & Kleinberg, J. (2003). The link prediction problem for social networks. In *Proceedings of the twelfth international conference on information and knowledge management* Association for Computing Machinery. (pp. 556–559). <https://doi.org/10.1145/956863.956972>
- Luo, Y., & Li, P. (2022). Neighborhood-aware scalable temporal network representation learning. In *Proceedings of the learning on graphs conference (log)*. <https://doi.org/10.48550/arXiv.2209.01084>
- Ma, Y., Guo, Z., Ren, Z., Tang, J., & Yin, D. (2020). Streaming graph neural networks. In *Proceedings of the 43rd international ACM SIGIR conference on research and development in information retrieval* Association for Computing Machinery. (p. 719–728). <https://doi.org/10.1145/3397271.3401092>
- Nguyen, G.H., Boaz Lee, J., Rossi, R.A., Ahmed, N.K., Koh, E., & Kim, S. (2018a). Dynamic network embeddings: From random walks to temporal random walks. In *2018 IEEE international conference on big data (big data)* (pp. 1085–1092). <https://doi.org/10.1109/BigData.2018.8622109>
- Nguyen, G.H., Lee, J.B., Rossi, R.A., Ahmed, N.K., Koh, E., & Kim, S. (2018b). Continuous-time dynamic network embeddings. In *Companion proceedings of the web conference 2018* (pp. 969–976).
- Pareja, A., Domeniconi, G., Chen, J., Ma, T., Suzumura, T., Kanezashi, H., Kaler, T., Schardl, T., & Leiserson, C.E. (2020). EvolveGCN: Evolving graph convolutional networks for dynamic graphs. In *Proceedings of the AAAI conference on artificial intelligence* (vol. 34). (pp. 5363–5370). <https://doi.org/10.1609/aaai.v34i04.5984>
- Poli, M., Massaroli, S., Park, J., Yamashita, A., Asama, H., & Park, J. (2019). Graph neural ordinary differential equations. *arXiv preprint arXiv:1911.07532*.
- Rossi, E., Chamberlain, B., Frasca, F., Eynard, D., Monti, F., & Bronstein, M. (2020). Temporal graph networks for deep learning on dynamic graphs. In *ICLR workshop on graph representation learning*.
- Sankar, A., Wu, Y., Gou, L., Zhang, W., & Yang, H. (2020). DySAT: Deep neural representation learning on dynamic graphs via self-attention networks. In *Proceedings of the 13th international conference on web search and data mining* Association for Computing Machinery. (p. 519–527). <https://doi.org/10.1145/3336191.3371845>
- Souza, A., Mesquita, D., Kaski, S., & Garg, V. (2022). Provably expressive temporal graph networks. *Advances in Neural Information Processing Systems*, 35, 32257–32269.
- Su, J., Zou, D., & Wu, C. (2024). Pres: Toward scalable memory-based dynamic graph neural networks. <https://arxiv.org/abs/2402.04284>.
- Tancik, M., Srinivasan, P., Mildenhall, B., Fridovich-Keil, S., Raghavan, N., Singhal, U., Ramamoorthi, R., Barron, J., & Ng, R. (2020). Fourier features let networks learn high frequency functions in low dimensional domains. 33, 7537–7547. https://proceedings.neurips.cc/paper_files/paper/2020/file/55053683268957697aa39fba6f231c68-Paper.pdf.
- Tang, J., & Wang, K. (2018). Personalized top-n sequential recommendation via convolutional sequence embedding. In *Proceedings of the eleventh ACM international conference on web search and data mining* (pp. 565–573).
- Thorpe, M., Nguyen, T.M., Xia, H., Strohmer, T., Bertozzi, A., Osher, S., & Wang, B. (2022). Grand + +: Graph neural diffusion with a source term. In *International conference on learning representations*.
- Tian, Y., Qi, Y., & Guo, F. (2024). FreeDyG: Frequency enhanced continuous-time dynamic graph model for link prediction. In *International conference on learning representations (ICLR)*. Poster <https://github.com/Tianxzzz/FreeDyG>.
- Trivedi, R., Farajtabar, M., Biswal, P., & Zha, H. (2019). DYREP: Learning representations over dynamic graphs. In *International conference on learning representations (ICLR)*.
- Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A.N., Kaiser, Ł., & Polosukhin, I. (2017). Attention is all you need. In *Proceedings of the 31st international conference on neural information processing systems (neurIPS)* Long Beach, CA, USA: Curran Associates Inc. (pp. 6000–6010). <https://doi.org/10.48550/arXiv.1706.03762>
- Veličković, P., Cucurull, G., Casanova, A., Romero, A., Liò, P., & Bengio, Y. (2018). Graph attention networks. In *International conference on learning representations (ICLR)*.

- Wang, X., Lyu, D., Li, M., Xia, Y., Yang, Q., Wang, X., Wang, X., Cui, P., Yang, Y., Sun, B. et al. (2021a). APAN: Asynchronous propagation attention network for real-time temporal graph embedding. In *Proceedings of the 2021 ACM SIGMOD international conference on management of data* ACM. (pp. 2628–2638). <https://doi.org/10.1145/3448016.3457274>
- Wang, Y., Wang, Y., Yang, J., & Lin, Z. (2021b). Dissecting the diffusion process in linear graph convolutional networks. In *Proceedings of the 35th conference on neural information processing systems (neurIPS)*. arXiv preprint arXiv:2102.10739.
- Wu, C.-Y., Ahmed, A., Beutel, A., Smola, A.J., & Jing, H. (2017). Recurrent recommender networks. In *Proceedings of the tenth ACM international conference on web search and data mining* (pp. 495–503).
- Xu, D., Ruan, C., Korpeoglu, E., Kumar, S., & Achan, K. (2020). Inductive representation learning on temporal graphs. In *International conference on learning representations (ICLR)*.
- Yu, B., Li, M., Zhang, J., & Zhu, Z. (2019). 3d graph convolutional networks with temporal graphs: A spatial information free framework for traffic forecasting. *arXiv preprint arXiv:1903.00919*.
- Yu, L., Sun, L., Du, B., & Lv, W. (2023). Towards better dynamic graph learning: New architecture and unified library. In *Proceedings of the 37th conference on neural information processing systems (neurIPS)*. Camera Ready Version. <https://doi.org/10.48550/arXiv.2303.13047>
- Zhang, M., & Chen, Y. (2018). Link prediction based on graph neural networks. In *Advances in neural information processing systems*. Curran Associates, Inc. (vol. 31). https://proceedings.neurips.cc/paper_files/paper/2018/file/53f0d7c537d99b3824f0f99d62ea2428-Paper.pdf.
- Zhang, Y., Liao, Y., Sun, Y., Jin, Y., Zheng, X., Xiong, Y., & Zhu, Y. (2023). Tiger: Temporal interaction graph embedding with restarts. In *Proceedings of the ACM web conference 2023 (WWW '23)* (pp. 1692–1702). <https://doi.org/10.1145/3543507.3583433>
- Zhang, Y., Xiong, Y., Li, D., Shan, C., Ren, K., & Zhu, Y. (2021). CoPE: Modeling continuous propagation and evolution on interaction graph. In *Proceedings of the 30th ACM international conference on information & knowledge management* Association for Computing Machinery. (p. 2627–2636). <https://doi.org/10.1145/3459637.3482419>